Лабораторная работа №7.
Обработка данных Web-форм с помощью алгоритмических конструкций JavaScript

Цель работы: изучить основы синтаксиса оператора if-else и оператора switch, циклов for, while, применить полученные знания для написания сценариев на JavaScript.

Теоретический материал

При составлении программ часто необходимо выполнение различных действий в зависимости от результатов проверки некоторых условий. Для организации ветвлений используются операторы if / else и оператор switch.

Оператор if / else

Основной синтаксис оператора if/else следующий:

```
if(условие){
  // код, который выполняется, когда справедливо условие if
} else {
  // код, который выполняется, когда условие if ложно
}
```

Необходимо отметить открывающую и закрывающую скобки, { и }, в приведенном выше коде. Открывающая скобка сообщает коду, где начинается блок кода, а закрывающая скобка указывает коду, где блок заканчивается. Поэтому все между { и } выполняется как часть оператора if. Необходимо отметить, что закрывающая скобка оператора if размещается непосредственно перед ключевым словом "else". Оператор else имеет свой собственный набор скобок и свой собственный блок для выполнения.

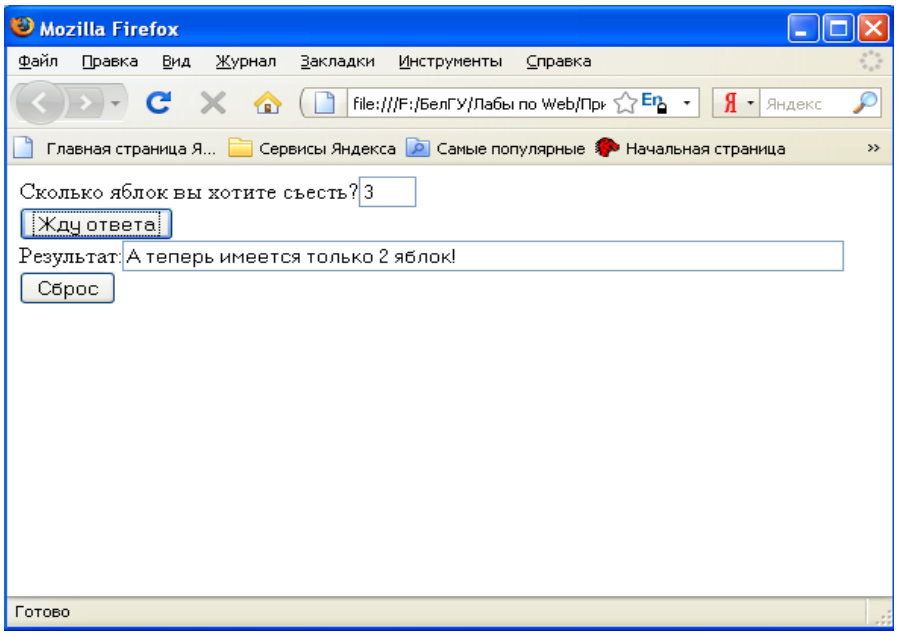
Две косые черты // в приведенном примере говорят коду, что здесь находится комментарий.

Пример. Требуется написать сценарий, который запрашивает у пользователя кол-во яблок, которое пользователь желает съесть (имеющееся кол-во яблок -5). Если пользователь вводит число больше 5-ти, то выводится сообщение о том, что имеется, лишь 5 яблок, поэтому нельзя съесть большее кол-во. Если пользователь вводит число меньше 5-ти, то выводится сообщение о количестве оставшихся яблок.

Ниже приведена реализация сценария на JS для приведенного примера.

```
<html>
<body>
<form name="form1">
<script type="text/JavaScript">
<!--
function def(obj)
{var eaten=Number(obj.eat.value);
var apples = 5;
d=document;
var s="";
```

```
if (eaten > 5) {
  s='Простите, но имеется только 5 яблок.  Вы не можете съесть ' +
eaten + ' яблок!';
} else {
  apples    -= eaten;
  s='А теперь имеется только ' + apples + ' яблок!';
}
obj.res.value=s;
}
//-->
</script>
Сколько яблок вы хотите съесть? <input type="text" size =2
name="eat"> <br>
<input type="button" value="Жду ответа" onClick="def(form1)">
<br>
Результат: <input type="text" size =75 name="res"><br>
<input type="reset">
</form>
</body>
</html>
```



Оператор switch

В тех случаях, когда при решении задачи требуется выбрать один вариант действия из нескольких возможных, удобно воспользоваться оператором switch.

Синтаксис оператора switch:

```
switch(B) {
  case L1:S1;
  case L2:S2;
```

```
  ..................
  case Ln:Sn;
  default:
}
```

где B-выражение; L1, L2,…, Ln – литералы; S1, S2, …,Sn,S-операторы.

Выполнение переключателя происходит так: вычисляется значение выражения B. Если значение B равно L1, выполняется оператор S1, а затем все остальные операторы, либо до первого оператора break, либо до конца оператора *switch*. Если значение B равно L2, выполняется оператор S2, а затем все остальные операторы, либо до первого оператора break, либо до конца оператора switch и т.д. Если же значение B не равно ни одному из значений L1, L2, …,Ln выполняются операторы S. Часть *default:S* может отсутствовать, тогда оператор имеет вид:

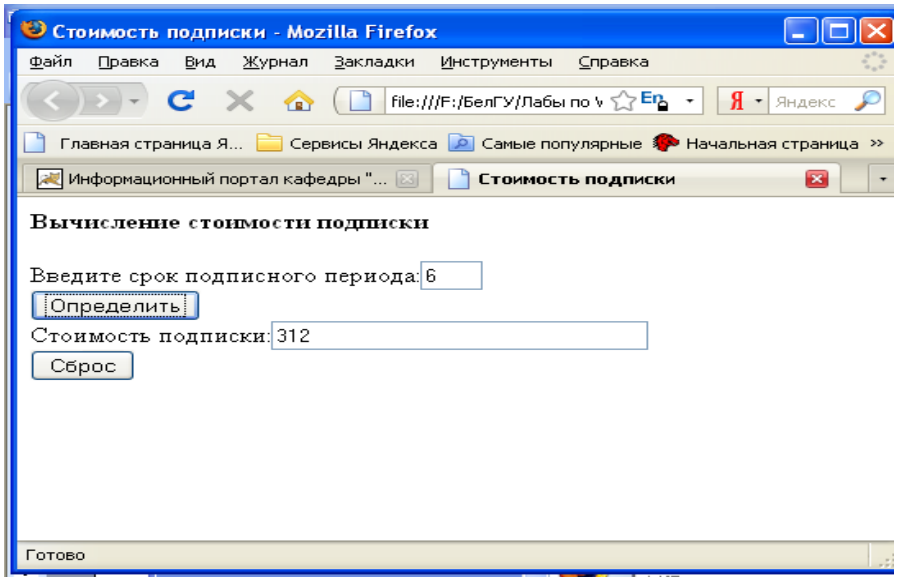
```
switch(B) {
  case L1:S1;
  case L2:S2;
  ..................
  case Ln:Sn;
}
```

Пример. Предположим, что с помощью анкеты, размещенной на Web-странице, пользователь хочет подписаться на электронное издание. Стоимость редакционной подписки зависит от срока подписного периода. Если выбирается 1 месяц, то стоимость будет 60 рублей, 3 месяца – 168 рублей, 6 месяцев -312 рублей, 12 месяцев – 540 рублей. Создадим документ, в котором пользователь вводит срок подписного периода (1, 3, 6, 12 месяцев), в зависимости от которого определяется стоимость подписки. В данном сценарии будем использовать оператор switch.

Ниже приведена реализация сценария на JS для приведенного примера.

```
<html>
<head>
<title>Стоимость подписки
</title>
<script type="text/JavaScript">
<!--
function def(obj){
  var n=Number(obj.num.value);
  d=document;
  var s="";
  switch(n){
       case 1:s=60;break;
       case 3:s=168;break;
       case 6:s=312;break;
       case 12:s=540;break;
       default:s="неверен срок подписного периода";
```

```
  }
obj.res.value=s;
}
//-->
</script>
</head>
<body>
<h4>Вычисление стоимости подписки</h4>
<form name="form1">
Введите срок подписного периода:<input type="text" size =2 name="num"><br>
<input type="button" value=Определить onClick="def(form1)"><br>
Стоимость подписки:<input type="text" size =34 name="res"><br>
<input type="reset">
</form>
</body>
</html>
```



Надо отметить, что в JavaScript можно задавать условные выражения. Условное выражение может иметь одну из двух значений, основанных на условии. Синтаксис:

(Условие)? Val1: val2

Если условие истинно, то выражение имеет значение val1, иначе имеет значение val2.

Например,

status = (age >= 18) ? "adult" : "minor"

Это утверждение присваивает значение "adult" переменной status, если age равно 18 или больше чем 18. Иначе, приписывает значение "minor" переменной status.

Оператор for

Создает цикл с тремя необязательными выражениями, заключенными в круглых скобках и разделенными точкой с запятой, сопровождаемыми блоками утверждений, выполненных внутри цикле:

```
for (initialExpression;condition;updateExpression) {
  ...строки кода...
}
```

где

- Выражение **initialExpression** используется для инициализации переменной счетчика, которую можно создать с помощью оператора **var**. Это выражение необязательно.
- Выражение **condition,** которое вычисляется на каждом проходе через цикл. Если это условие истинно, то выполняются условия внутри цикла. Это условие необязательно. Если оно опущено, то условие всегда считается равным истинному, тогда цикл продолжается до ошибки или до оператора **break**.
- Выражение **updateExpression**, вообще используется для изменения значения переменной счетчика. Оно тоже является необязательным. В этом случае можно обновлять значение переменной счетчика внутри цикла.

Пример. Оператор for создает цикл, продолжающийся до тех пор, пока переменная счетчика i меньше чем девять, увеличиваю i на один после каждого прохода через цикл.

```
for (var i = 0; i < 9; i++) {
  n += i
  myfunc(n)
}
```

Следующий **пример** создает цикл for, продолжающийся до наступления ошибки или выполнения оператора break. Переменная счетчика увеличивается на при каждом проходе цикла:

```
for (var i = 0; ; i+=2) {
  ...строки кода...
 }
```

Оператор while

Оператор while — это оператор цикла, который повторяет цикл, пока значение condition есть истинно (true). Как только значение condition становится ложным (false), то управление переходит к первому оператору после фигурной скобки, закрывающей тело цикла while:

```
while (condition) {
  ...statment...
}
```

Пример. Следующий цикл while повторяется, пока переменная n меньше чем три. Каждая итерация увеличивает n и прибавляет его к x. Следовательно, x и n принимают следующие значения:

- После первой итерации: x = 1 , n = 1.

- После второго итерации: x = 2 , n = 3.
- После третьего итерации: x = 3 , n = 6.

После завершения третьей итерации, условие n < 3 становится ложным, следовательно, цикл прекращается.

```
n = 0
x = 0
while( n < 3 ) {
  n ++;
  x += n
}
```

Следующий **пример** проверяет наличие в строке определенного символа и прекращает поиск по окончании перебора всех символов строки или после того, как находит искомый символ.

```
var  found=false
n=0
while (n<=searhcString.length II !found) {
  if (searhcString.charAt[n]=="?")
     found = true
  else
     n++;
 }
```

Оператор do . . . while

Оператор do . . . while — это оператор цикла с пост условием, который повторяет цикл, пока значение condition есть истинно (true). Как только значение condition становится ложным (false), то управление переходит к первому оператору после ключевого слова while:

```
do {
  ...statment...
}
while (condition)
```

Оператор break

Оператор. Прерывает текущий цикл for и while, передавая управление первому оператору после цикла.

Пример. Следующий пример добавляет элемент к форме в предположении, что все элементы содержат числовые значения. Если встречается значение "0", цикл прекращается.

```
 function CheckValues(form) {
   var total
   for (I=0; I<=form.elements.length; I++) {
      if (element[I].value="0") {
        break;
      }
      else {
        total +=I;
        document.write("Текущая строка: "+total);
```

```
      }
    }
    return total
  }
```

Оператор continue

Передает управление оператору проверки истинности условия в цикле while и оператору обновления значения счетчика в цикле for. Важное отличие от оператора break заключается в том, что оператор continue не прерывает цикл, а делает следующее.

- В цикле **while**, это переходит назад к условию.
- В цикле **for**, это переходит к модернизированному выражению.

Пример. Следующий цикл while повторяется, пока переменная n меньше чем три. Каждая итерация увеличивает n и прибавляет его к x. Следовательно, x и n принимают следующие значения:

- После первой итерации: x = 1 , n = 1.
- После второго итерации: x = 2 , n = 3.
- После третьего итерации: x = 3 , n = 6.

После завершения третьей итерации, условие n < 3 становится ложным, следовательно, цикл прекращается.

```
n = 0
x = 0
 while (n < 3) {
   n ++;
   x += n
 }
```

Следующий **пример** проверяет наличие в строке определенного символа и прекращает поиск по окончании перебора всех символов строки или после того, как находит искомый символ.

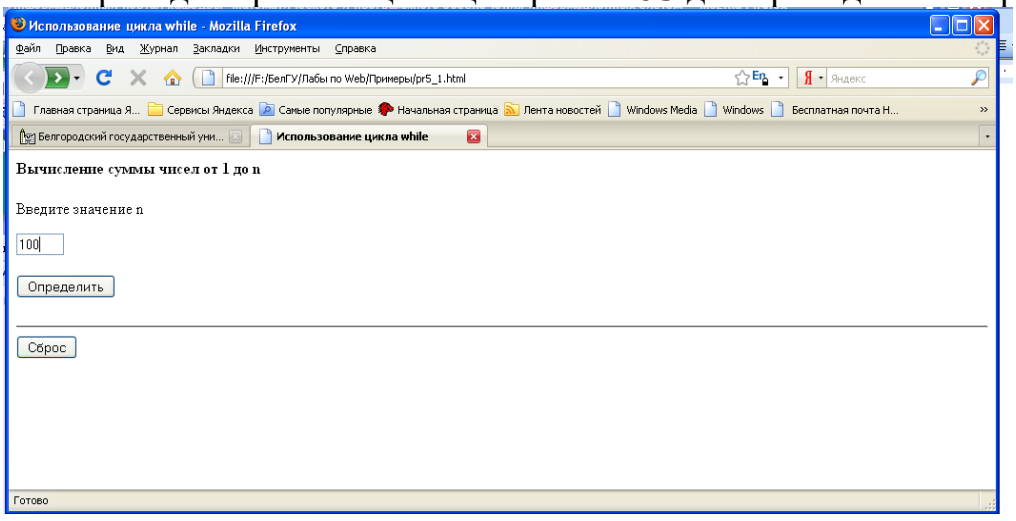
```
var found=false
n=0
while (n<=searhcString.length II !found) {
  if (searhcString.charAt[n]=="?")
    found = true
  else
    n++;
}
```

Пример. Вычислить суммы чисел от 1 до n.

```
<html>
<head>
  <title> Использование цикла while </title>
  <script type="text/JavaScript">
<!-- //
function add(obj) {
  var n=Number(obj.num.value);
```

```
   var i=0;
   var result=0;
   document.write("Результат расчета :"+"<br>");
   document.write(0);
   while(i<n) {
    if (i%25 == 0 && i>0) document.write("<br>");
    result+= ++i;
    document.write("+"+i);
   }
   document.write("="+result);
}
//-->
</script>
</head>
<body>
 <h4>Вычисление суммы чисел от 1 до n</h4>
 <p> Введите значение n</p>
 <form name ="form1">
 <input type="text" name="num" size =4> <br><br>
 <input type="button" value="Определить" onClick = "add(form1)">
 <br><br>
 <hr>
 <input type="reset">
 </form>
</body>
</html>
```

Ниже приведена реализация сценария на JS для приведенного примера.



Общая постановка задачи

1. Выполнить два индивидуальных задания по варианту. (Переменные должны быть только числового типа. Нельзя использовать строковые переменные).
2. Поместить разработанный **JS**-код сценария на FTP-сервере.

3. Составить электронный отчет о проделанной работе.

Список индивидуальных данных

Задание №1

1. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить целое число, во второе поле выводится строка «четное число», если введенное число является четным и «нечетное» в обратном случае.

2. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число, во второе поле выводится квадратный корень из числа, в случае если введенное число отрицательно, то выводится сообщение «действительного корня не существует».

3. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число, во второе поле выводится модуль введенного числа.

4. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить число от 1 до 12 , во второе поле выводится название месяца, с порядковым номером равным введенному числу, в случае ввода несуществующего номера месяца выводится сообщение - «введите число от 1 до 12».

5. Создать документ с формой, содержащей 2 поля для ввода и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить название дня недели , во второе поле выводится с порядковый номер введенного дня, в случае ввода несуществующего дня недели выводится сообщение - «нет такого дня».

6. Создать документ с формой, позволяющей вводить коэффициенты квадратного уравнения и выводить корни уравнения, либо сообщение о том, что действительных корней не существует.

7. Создать документ с формой, позволяющей задать уравнение параболы, и определить пересекает ли она ось X или нет.

8. Создать документ с формой, содержащей 2 текстовых поля и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид. В первое поле пользователь должен вводить название города (задать в программе 5 городов), во второе поле выводится температура воздуха в городе, если данных о температуре введенного города нет, то выводится сообщение «таких данных нет».

9. Создать документ с формой, содержащей 1 список, 2 текстовых поля и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид, для оформления заказа покупки в Интернет-магазине. В магазине продаются ручки по 6 руб., карандаши – по 3 руб. и фломастеры – по 15 руб. Пользователю предоставляется возможность выбрать из списка

товаров какой-то один и ввести количество товара, которое он хочет купить. После составления заказа, в поле формы выводится стоимость заказа.

10. Создать документ с формой, содержащей 1 список,1 текстовую область и 2 кнопки: для подтверждения ввода и перевода формы в первоначальный вид, для вывода информации о члене вашей группы. Пользователь должен выбрать в списке фамилию члена вашей группы, а в текстовой области формы выводится информация о дате рождения этого человека (или какая-то другая информация).
11. Написать программу на HTML и JavaScript для следующего задания: определить, каким является треугольник, заданный координатами своих вершин: равносторонним.
12. Написать программу на HTML и JavaScript для следующего задания: упорядочить 3 числа по убыванию.
13. Написать программу на HTML и JavaScript для следующего задания: найти максимальную цифру в записи данного трехзначного числа.
14. Написать программу на HTML и JavaScript для следующего задания: определить, находится ли точка с координатами (x, y) в I или III четверти и внутри круга данного радиуса r с центром в начале координат.
15. Написать программу на HTML и JavaScript для следующего задания: Даны координаты вершин треугольника: (ax, ay), (bx, by), (cx, cy). Определить, лежит ли данный треугольник внутри окружности радиуса R с центром в точке (x0, y0).
16. Написать программу на HTML и JavaScript для следующего задания: Даны действительные числа x, y, z. Выяснить, существует ли треугольник с длинами сторон x, y, z.
17. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 3 вещественных чисел. Определить количество введенных четных чисел.
18. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 3 вещественных чисел. Определить сумму введенных нечетных чисел.
19. Написать программу на HTML и JavaScript для следующего задания: Даны действительные числа a, b, c, d. Если они упорядочены по невозрастанию, то каждое число заменить наибольшим из них; если они упорядочены по неубыванию, то числа оставить без изменения; в противном случае все числа заменить их квадратами.
20. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 3 вещественных чисел. Определить количество введенных нулей.
21. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 5 вещественных чисел. Определить число, предшествующее первому из введенных минимальных значений.
22. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 3 вещественных чисел. Определить количество максимальных значений.

23. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 3 вещественных чисел. Определить максимальное число.

24. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 6 вещественных чисел. Вычислите сумму тех из них, которые нечетны и отрицательны.

25. Написать программу на HTML и JavaScript для следующего задания: на экране в форме вводятся 6 вещественных чисел. Определить сколько раз в последовательности меняется знак при переходе к следующему элементу.

Задание №2

1. Написать программу на HTML и JavaScript, которая определяет, является ли заданное натуральное число палиндромом.
2. Создать документ, с формой, позволяющей пользователю вводить число и степень, в которую необходимо возвести это число. В форму выводится результат возведения в степень (использовать циклы).
3. Создать документ, с формой, позволяющей найти количество цифр, во введенном пользователем числе.
4. Создать документ, с формой, позволяющей найти сумму цифр, во введенном пользователем числе.
5. Создать документ, с формой, которая позволяет узнать является ли число, введенное пользователем простым.
6. Создать документ, с формой, которая позволяет узнать сумму целых чисел в некотором диапазоне. Диапазон вводится пользователем.
7. Создать документ, с формой, которая позволяет узнать количество четных чисел в некотором диапазоне. Диапазон вводится пользователем.
8. Создать документ, с формой, которая позволяет узнать количество чисел, которые делятся на 7 в некотором диапазоне. Диапазон вводится пользователем.
9. Создать документ, в котором цвет фона (свойство document.bgColor) меняется спустя некоторое время (использовать циклы, в качестве счетчика времени).
10. Написать программу на HTML и JavaScript для следующего задания: определите наибольший общий делитель 2-х заданных чисел n и m. Согласно алгоритму Евклида: если m=0, НОД(n,m)=НОД(n,0)=n. В остальных случаях НОД(n,m)=НОД(m,n%m).
11. Написать программу на HTML и JavaScript для следующего задания: определите, является ли введенное число совершенным. Совершенным называется число n, равное сумме своих делителей, не считая самого числа.
12. Написать программу на HTML и JavaScript для следующего задания: найти максимальную цифру в записи заданного числа.
13. Написать программу на HTML и JavaScript для следующего задания: определить являются ли числа взаимно простыми. Взаимно простые числа не имеют общих делителей кроме 1.

14. Написать программу на HTML и JavaScript для следующего задания: проверить является ли число n степенью числа 3.
15. Написать программу на HTML и JavaScript для следующего задания: переставить первую и последнюю цифры числа n.
16. Написать программу на HTML и JavaScript для следующего задания: определить являются ли два введенных числа близнецами.Два натуральных числа являются близнецами, если они оба просты и разница между ними равна 2.
17. Написать программу на HTML и JavaScript для следующего задания: определите, являются ли два введенных числа дружественными. Числа называются дружественными, если каждое из них равно сумме делителей другого не считая самого числа.
18. Написать программу на HTML и JavaScript для следующего задания: определить все числа из заданного диапазона, которые образуют пару дружественных чисел. Числа называются дружественными, если каждое из них равно сумме делителей другого не считая самого числа.
19. Написать программу на HTML и JavaScript для следующего задания: вывести n первых чисел Фибоначчи. Числа Фибоначчи это последовательность таких чисел, где первые два числа равны 1, а каждый следующий элемент представляет собой сумму двух предыдущих.
20. Написать программу на HTML и JavaScript для следующего задания: вычислите сумму чисел из заданного диапазона, которые кратны 5. Границы диапазона задаются пользователем.
21. Написать программу на HTML и JavaScript для следующего задания: определить по заданному натуральному числу n значение n!! Напомним, что n!!=1*3*…*n, если n-нечетно и n!!=2*4*…*n, если n-четно.
22. Написать программу на HTML и JavaScript для следующего задания: определить сумму первых n элементов последовательности, где i-й элемент записан в виде: $\frac{x^i}{i!}$. x и n определяет пользователь.
23. Написать программу на HTML и JavaScript для следующего задания: определить наименьшее общее кратное 2 чисел. Заметим, что НОК(n,m)=n*m/НОД(n,m).
24. Написать программу на HTML и JavaScript для следующего задания: определите все простые трехзначные числа.
25. Написать программу на HTML и JavaScript, которая «переворачивает» заданное натуральное число.